Stack

Valid Parentheses	**→ Easy**
Simplify Path	**→ Medium**
Min Stack	**→ Medium**
Evaluate Reverse Polish Notation	**→ Medium**
Basic Calculator	**→ Hard**

20. Valid Parentheses

Given a string `s` containing just the characters `'('`, `')'`, `'{'`, `'}'`, `'['` and `']'`, determine if the input string is valid.

An input string is valid if:

1. Open brackets must be closed by the same type of brackets.
2. Open brackets must be closed in the correct order.
3. Every close bracket has a corresponding open bracket of the same type.

```
class Solution {  // 1 way
public:
    bool isValid(string s) {
        stack<char> st;

        for(char ch:s) {
            if(st.empty() || ch == '(' || ch == '{' || ch == '[') {
                st.push(ch);
                continue;
            }

            if(ch == ')') {
                if(st.top() == '(')
```

```
                st.pop();
            else
                return false;
        } else if(ch == '}') {
            if(st.top() == '{')
                st.pop();
            else
                return false;
        } else if(ch == ']') {
            if(st.top() == '[')
                st.pop();
            else
                return false;
        }
    }

    return st.empty();
  }



};

// oR
class Solution {  // 2 way
public:
  bool isValid(string s) {
    stack<char> st;

    for(char ch:s) {
        if (ch == '(')
            st.push(')');
        else if (ch == '{')
            st.push('}');
        else if (ch == '[')
            st.push(']');
        else if (st.empty() || st.top() != ch)
```

```
            return false;
        else {
            st.pop();
        }
    }

    return st.empty();

  }
};
```

20. Valid Parentheses

i/p: s = "()[]{}"

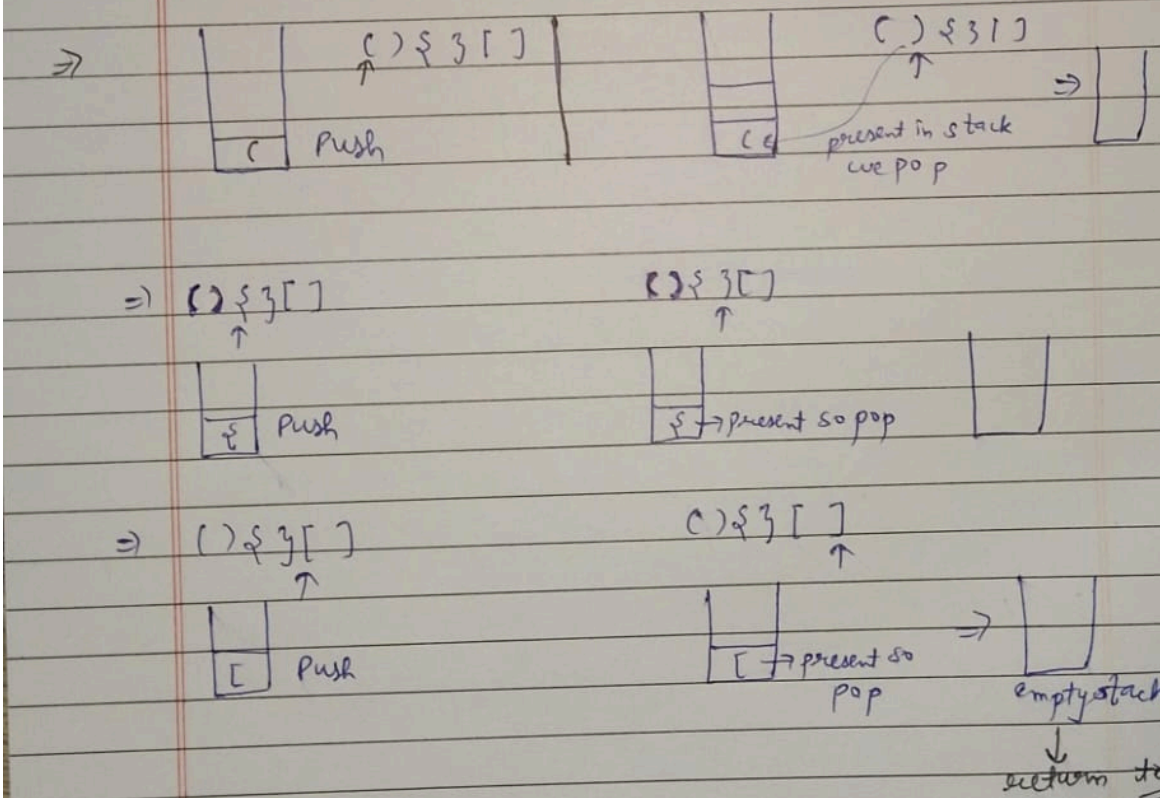
o/p: True

Approach 1

(
{
[

For open brackets we push in stack

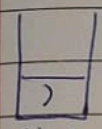
For closing brackets we will match with open brackets whether it is present in stack or not if present then we pop



Approach 2

⇒ ( ) { } [ ]

↑
i

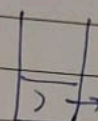


Push close bracket

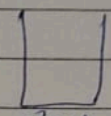
Stack

( ) { } [ ]

↑
i



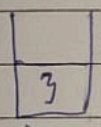
→ Top has this close bracket so we pop



Stack

⇒ ( ) { } [ ]

↑
i

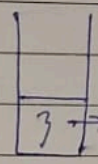


Push close bracket

Stack

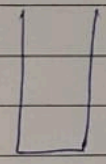
( ) { } [ ]

↑



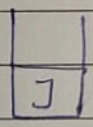
→ pop

⇒



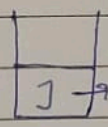
⇒ ( ) { } [ ]

↑
i



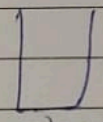
( ) { } [ ]

↑



→ Pop

→



stack is empty

155. Min Stack

Design a stack that supports push, pop, top, and retrieving the minimum element in constant time.

Implement the `MinStack` class:

- `MinStack()` initializes the stack object.
- `void push(int val)` pushes the element `val` onto the stack.
- `void pop()` removes the element on the top of the stack.
- `int top()` gets the top element of the stack.
- `int getMin()` retrieves the minimum element in the stack.

You must implement a solution with `O(1)` time complexity for each function.

```
class MinStack {  // 1 way
public:
    stack<pair<int ,int>> s;

    MinStack() {

    }

    void push(int val) {
        if(s.empty()){
            s.push({val , val});
        }
        else{
            int minVal = min(val , s.top().second);
            s.push({val , minVal});
        }

    }
```

```
    void pop() {
        s.pop();

    }

    int top() {
        return s.top().first;

    }

    int getMin() {
        return s.top().second;

    }
};

// OR

class MinStack {  // 2 way
public:
    stack<long long int> s;
    long long int minVal;
    MinStack() {

    }

    void push(int val) {
        if(s.empty()){
            s.push(val);
            minVal =val;
        }
        else{
            if(val< minVal){
                s.push((long long)2 * val - minVal);
                minVal = val;
```

```
        }
        else{
            s.push(val);
        }
    }

}

void pop() {
    if(s.top() < minVal){
        minVal =2 * minVal -s.top();
    }
    s.pop();

}

int top() {
    if(s.top() < minVal){
        return minVal;
    }
    return s.top();

}

int getMin() {
    return minVal;

}
};
```

Date : / /
Page No.

Design : Min Stack

all operations should be constant time

Push -2

Push 0

Push -3

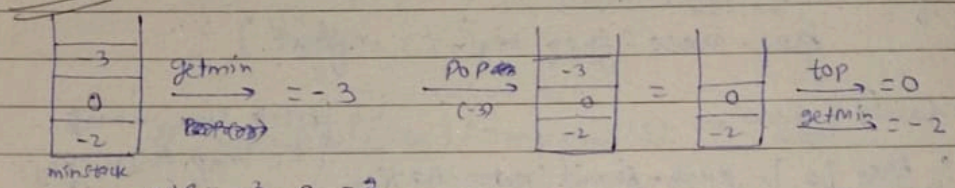
getMin → returen minimum element from all elements which are present in stack

pop

top

getMin

1st way



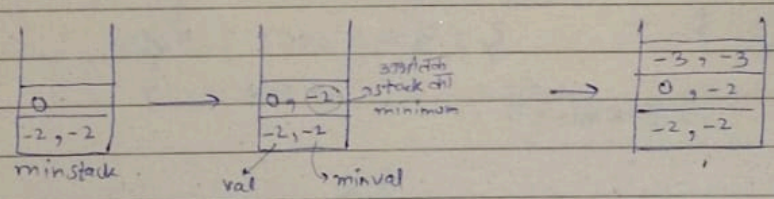
o/p = -3, 0, -2

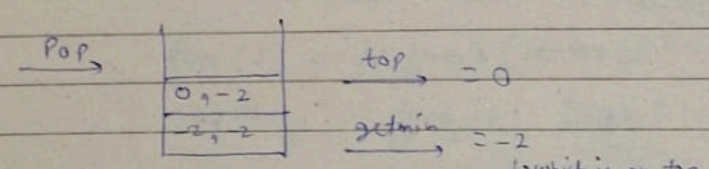
to return for constant time → the min value should be present in stack.

So, we take

Stack <pair <val, minVal>>

↳ अभी तक stack में जो भी min value है उसको store करेंगे






```
public:
stack<pair<int, int>> s;
minstack() {
}
```

val minVal

Push ⇒

```
if (s.empty())
{ s.push({val, val});
}
```

→ if stack is empty ie 1st element when push in stack so its the value & minVal

Date : / /
Page No.

else { → when stack is not empty

int minVal = min(Val , s.top().second);

-3 , -3
0 , -2
-2 , -2

-3 and -2 is compare

minval

top

and those minimum put on top of stack.

min(-3 , -2)

= -3

s.push({ val , minval });

```
void pop() {
    s.pop();
}

int top() {
    return s.top().first;
}

int getMin() {
    return s.top().second;
}

};
```

ई२ element के साथ ई। ई।
minval store कर रहे हैं।

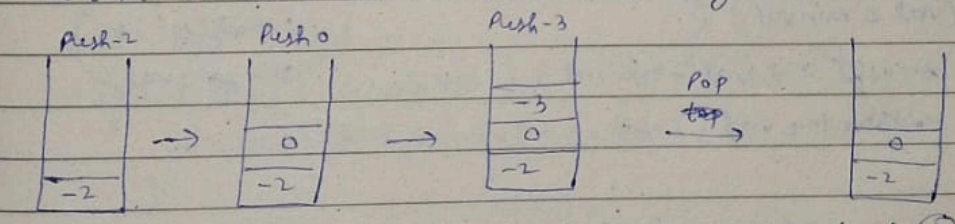
for n push operations then space comp. = O(2*n)

because on every push operation we push double data into stack

val minval

2nd way

we take variable = minval → which store global minValue



minVal=-2 minVal=-2 minVal=-3 minVal = -3 ✗

we don't go back

So we need other method to go back from new minval to old minval

old minval

new minval

minVal = ~~-2~~ -3

minval = -2

Date: / /
Page No.

Push ⇒ val (current) < minval → तो stack को minval को global minval से change करना होगा।

Push val' in the stack → $\boxed{val' = 2 * val - minval}$ → अभी तक जो भी minval चल रही है।

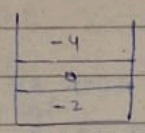
(val' → new val, minval → old → previous min value)

Push (-3) ⇒ $= 2 \times (-3) - (-2)$

$= -6 + 2$

$= -4$

and update ⇒ minval = val (current value i.e (-3))



stack <int>

Pop

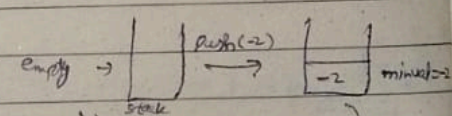
old minval = 2 * new minval - val'

= 2 * (-3) - (-4) ↳ which is push in stack

= -2 ↳ -2 is old minval

int getmin() → O(1)

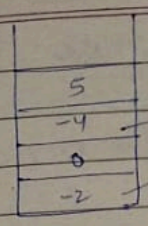
{ return minval; }




```
void push (val)
{
    if (val < minval) {
        s.push( 2 * val - minval )
        minval = val
    }
else ⇒ → (val >= minval)
{
    s. push (val) → (So only push our normal val)
}
```

→ if (s. Empty())
s. push (val) &
minval = val

Date: / /
Page No.



minval = ~~2~~ -3

val < minval (current) i.e -4 < -3

when val' is push तो val' क्या था

val' < minval

val' = 2 × val - minval

अब minval को change कर रहे हैं

means minval > val

= val + (new val - old minval) → greater value

-ve

= val + (-ve)

= -ve

< minval

less than

and this less than minval push in stack i.e (-4)

eg Push = -3

But stack में हमने (-4) push किया

which is result of this formula

val' = 2 * val - minval

minval = -3

-4 < -3

जो stack में push (-4) होती है वो हमारे minval (-3) से भी कम होती है।

-4<3 then

when we pop

again we apply

this formula to change our minval

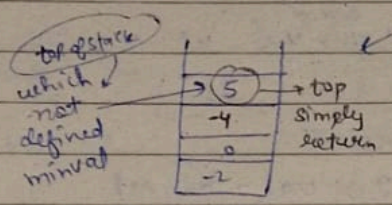
old minval = 2 * newminval - val'

~~to change our minval~~

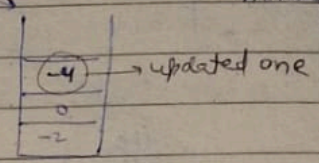
```
void pop() {
    if (s.top() < minval) → i.e -4 < -3 then
        minval = 2 * minval - s.top();  → update minval
    }
    s.pop()
}
```

Date: / /
Page No.

top(·) Two Cases



minval = -3

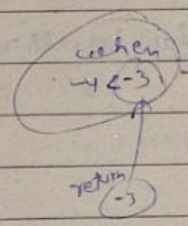


Stack के top पे formula से apply कर के change किया हुआ minval है।

i.e हमारे stack के top पे -3 होना चाहिए था but by applying Formula ⇒ val' = 2 × new val − old minval

we have this val' in top

But we need to return -3 तो हम stack के top वाली value return नहीं करते. In this case we return minval not the stack top value.



top() → if s.top() < minval → return minval
→ else → return s.top()

Space complex = O(n)
we use stack & globalvariable (minval)

71. Simplify Path

You are given an *absolute* path for a Unix-style file system, which always begins with a slash `'/'`. Your task is to transform this absolute path into its **simplified canonical path**.

The *rules* of a Unix-style file system are as follows:

- A single period `'.'` represents the current directory.
- A double period `'..'` represents the previous/parent directory.
- Multiple consecutive slashes such as `'//'` and `'///'` are treated as a single slash `'/'`.
- Any sequence of periods that does **not match** the rules above should be treated as a **valid directory or file name**. For example, `'...'` and `'....'` are valid directory or file names.

The simplified canonical path should follow these *rules*:

- The path must start with a single slash `'/'`.
- Directories within the path must be separated by exactly one slash `'/'`.
- The path must not end with a slash `'/'`, unless it is the root directory.
- The path must not have any single or double periods (`'.'` and `'..'`) used to denote current or parent directories.

Return the **simplified canonical path**.

Example :

Input: path = "/.../a/../b/c/../d/./"

Output: "/.../b/d"

Explanation:

`"..."` is a valid name for a directory in this problem.

```
//   1 way -------------------------------------
class Solution {
public:
    string simplifyPath(string path) {
```

```
        string token = "";

        stringstream ss(path);
        stack<string> st;

        while(getline(ss, token, '/')) {

            if(token == "" || token == ".") continue;

            if (token != "..")
                st.push(token);
            else if (!st.empty())
                st.pop();
        }

        string result = "";

        while(!st.empty()){ // add all the stack elements
            result="/"+st.top()+result;
            st.pop();
        }

        if(result.length()==0) // if no directory or file is present
            result="/"; // minimum root directory must be present in result

        return result;
    }
};


//------------------------------- 2 way -----------------------------------

class Solution {
public:
    string simplifyPath(string path) {
         string token = "";
```

```
        stringstream ss(path);
        vector<string> st;

        while(getline(ss, token, '/')) {

            if(token == "" || token == ".") continue;

            if (token != "..")
                st.push_back(token);
            else if (!st.empty())
                st.pop_back();
        }

        string result = "";

        for(auto &token : st) {
            result += "/" + token;
        }

        return result.empty() ? "/" : result;
    }
};
```

71. Simplify path

i/p "/a/./b/../../c/"

o/p "/c"

/ → root in linux
. → current directory
.. → return to current directory

a/b ⇒ a to b
a//b
a////b
multiple slash → means single slash

Stack

Approach 1 stack

path ⇒ "/a/./b/../../c/"

Tokenize in c++

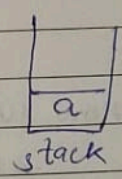
tokenize based on /

after tokenize

space

token ⇒ "a" "." "b" ".." ".." " " "c"

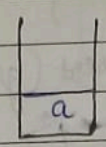
file name



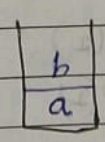
empty and in dot (.) ⇒ do nothing | (..) ⇒ Pop

⇒ a . b .. .. " " c

do nothing

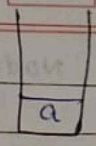


⇒ 'a' '.' 'b' '..' '..' " " 'c'



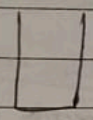
⇒ a • b .. .. " " c

↑ Pop (b)



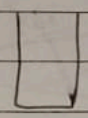
⇒ a • b .. .. " " c

↑ Pop(a)



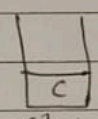
⇒ a • b .. .. " " c

↑ empty do nothing



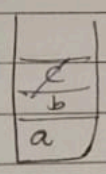
⇒ a • b .. .. " " c

↑ (Push)



stack

Eg



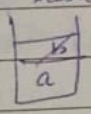
a|b|c

to prevent order

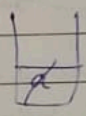
result = " ";

result = "(|" + st.top(); + result;

result = |c



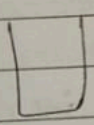
result = |b|c



= |a|b|c → o/p

result

⇒

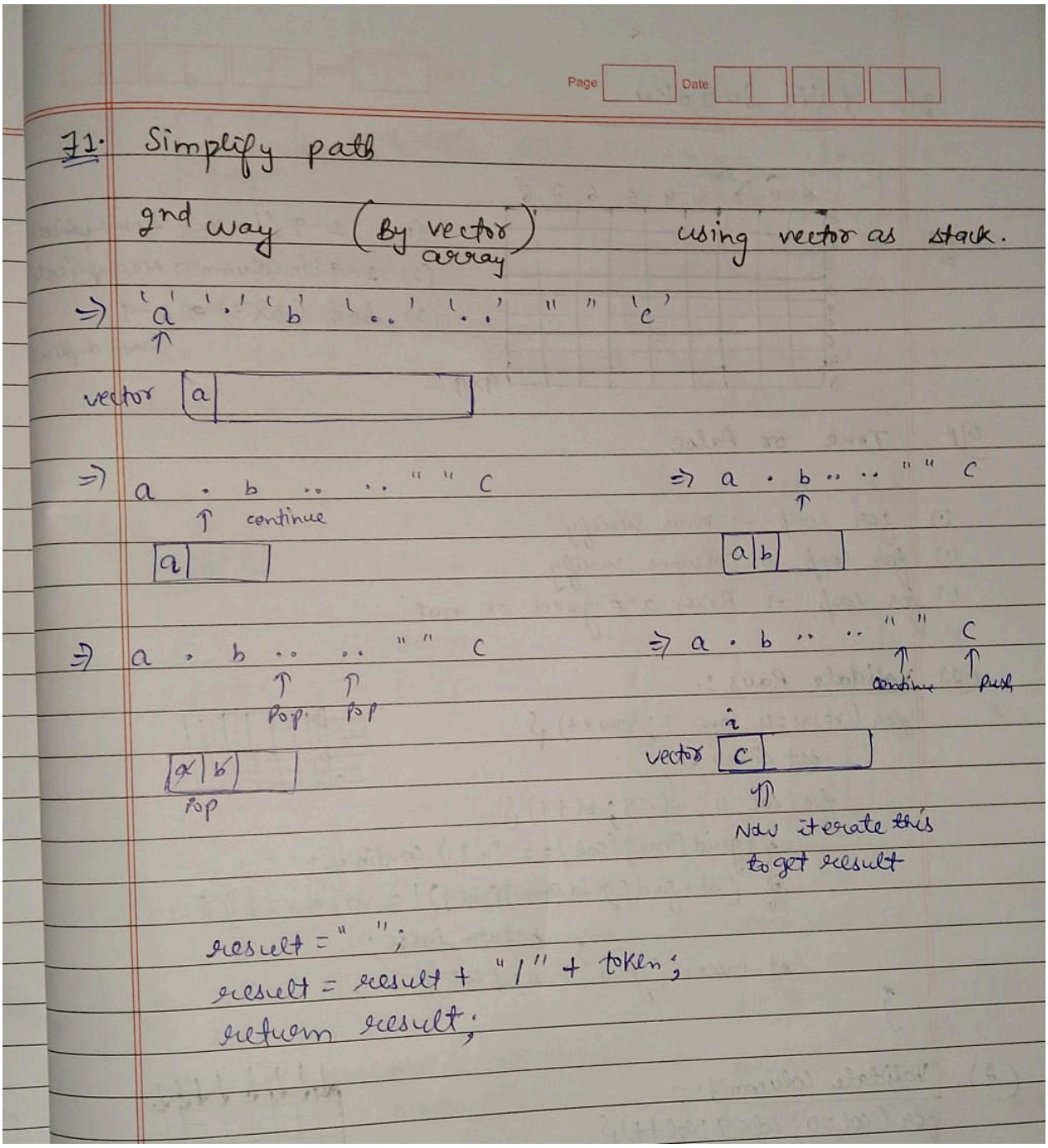


⇒ if stack is empty so return root ⇒ "|"

tokenize ⇒ while (getline (ss, token, '|'))

T.C = o(n)

S.C = o(n)



150. Evaluate Reverse Polish Notation

You are given an array of strings `tokens` that represents an arithmetic expression in a Reverse Polish Notation.

Evaluate the expression. Return *an integer that represents the value of the expression*.

Note that:

- The valid operators are `'+'`, `'-'`, `'*'`, and `'/'`.
- Each operand may be an integer or another expression.
- The division between two integers always **truncates toward zero**.
- There will not be any division by zero.
- The input represents a valid arithmetic expression in a reverse polish notation.
- The answer and all the intermediate calculations can be represented in a **32-bit** integer.

```
//      1 way

class Solution {
public:
    int operate(int &a, int &b, string &s) {
        if(s == "+")
            return a+b;

        if(s == "-")
            return a-b;

        if(s == "*")
            return (long)a*(long)b;

        if(s == "/")
            return a/b;

        return -1;
    }
    int evalRPN(vector<string>& tokens) {
        stack<int> st;
        int result = 0;
        for( string& s:tokens) {
            if(s == "+" || s == "-" || s == "*" || s == "/") {
```

```
            int b = st.top();
            st.pop();
            int a = st.top();
            st.pop();

            result = operate(a, b, s);
            st.push(result);
        } else {
            st.push(stoi(s)); // operand push
        }
    }
    return st.top();
  }
};

//          2 way  by lambda on unodered_map


class Solution {
public:
   int evalRPN(vector<string>& tokens) {
       stack<int> st;
    int result = 0;

    unordered_map<string, function<int (int, int)> > mp = {
        {"+", [](int a, int b) {return a + b; } },
        {"-", [](int a, int b) {return a - b; } },
        {"*", [](int a, int b) {return (long)a * (long)b; } },
        {"/", [](int a, int b) {return a / b; } },
    };

    for(const string& s:tokens) {
        if(s == "+" || s == "-" || s == "*" || s == "/") {
            int b = st.top();
            st.pop();
            int a = st.top();
```

```
            st.pop();

            result = mp[s](a, b);
            st.push(result);
        } else {
            st.push(stoi(s));
        }
    }
    return st.top();
  }
};
```

Imp

150. Evaluate Reverse Polish Notation

i/p tokens = { "2", "1", "+", "3", "*" } → strings

o/p 9

⇓

No. from -200 to 200

operators → +, -, *, /

⇒ $\underbrace{2, 1, +}_{2+1=3}$, 3, *

⇒ $\underbrace{2, 1, +}$, 3, *

3 * 3 = 9

past result

(1) use stack T.C = O(n), S.C = O(n)

⇒ "2", "1", "+", "3", "*"

i, i

1
2

stack

operand push in stack

⇒ "2", "1", "+", "3", "*"

i

1 → Pop
2 → Pop

2+1 = 3 → result push in stack

3

⇒ "2", "1", "+", "3", "*"

i, i

2 → Pop
3 → Pop

⇒ 3 * 3 = 9

9

→ top is result → return st.top();

(string)
stoi(" ") $\xrightarrow[\text{into}]{\text{convert}}$ integer

to avoid overflow in multiplication

(long)3 * long(3)

(2) using lambda on unordered_map

value is a function

unordered_map < string , function < int (int, int) > > mp

key — return — takes

= { { "+" , [ ] (int a , int b) { return a+b; } } ,

function

----- and so on

};

⇒ call this map

result = mp ["+"] (a, b)

(takes operand)

(+ function is call)

key	value
"+"	function < >